Análisis del software: vistas dinámicas

Proyecto formativo · GPS Guardián Escolar

Equipo del proyecto GPS Guardián Escolar

5 de noviembre de 2026

Versión documental: 2.5.7 · **Estado:** Estable

Tabla de contenido

- Propósito, frontera y actores
- Escenario AV-01: registro, verificación y autenticación
- Escenario AV-02: estudiante, acudiente y vinculación QR
- Escenario AV-03: asignar ruta y establecer ruta preferida
- Escenario AV-04: trayecto GPS y comunicación en tiempo real
- Escenario AV-05: ubicación obsoleta y notificación
- Escenarios AV-06 a AV-08: zonas seguras, mapas y notificaciones
- Matriz de condiciones excepcionales
- Diagrama de estados y pruebas de consistencia
- Criterios de aceptación de la vista dinámica
- Referencias y trazabilidad (APA 7.ª edición)

Resumen

Se analizan las interacciones entre los usuarios, el cliente Expo, los servicios Spring Boot, PostgreSQL y proveedores externos. Las secuencias se basan en los controladores y servicios del corte de main. El análisis diferencia estados implementados, condiciones de error y controles cuya ejecución aún debe comprobarse. Se corrige una inconsistencia histórica: el controlador de trayectos revisado **no declara PATCH /api/trips/{id}/status**; en este corte la cancelación documentada se realiza mediante DELETE /api/trips/{tripId} con baja lógica.

1. Propósito, frontera y actores

Objetivo: determinar responsabilidades, orden de mensajes, precondiciones, caminos alternativos y eventos observables para los procesos de registro, vinculación, rutas, viajes y alertas. **Alcance:** lógica descrita por el código consultado; la disponibilidad de recursos externos no se deduce de sus integraciones. **Actores:** acudiente autorizado, administrador, dispositivo infantil vinculado, plataforma de correos, Google Maps, plataforma Expo Push y operador técnico.

La frontera del producto es una API REST /api/** complementada por STOMP/WebSocket /ws. Las sesiones y privilegios se validan en el servidor; el cliente gestiona la experiencia de usuario, sin sustituir el control de autorización.

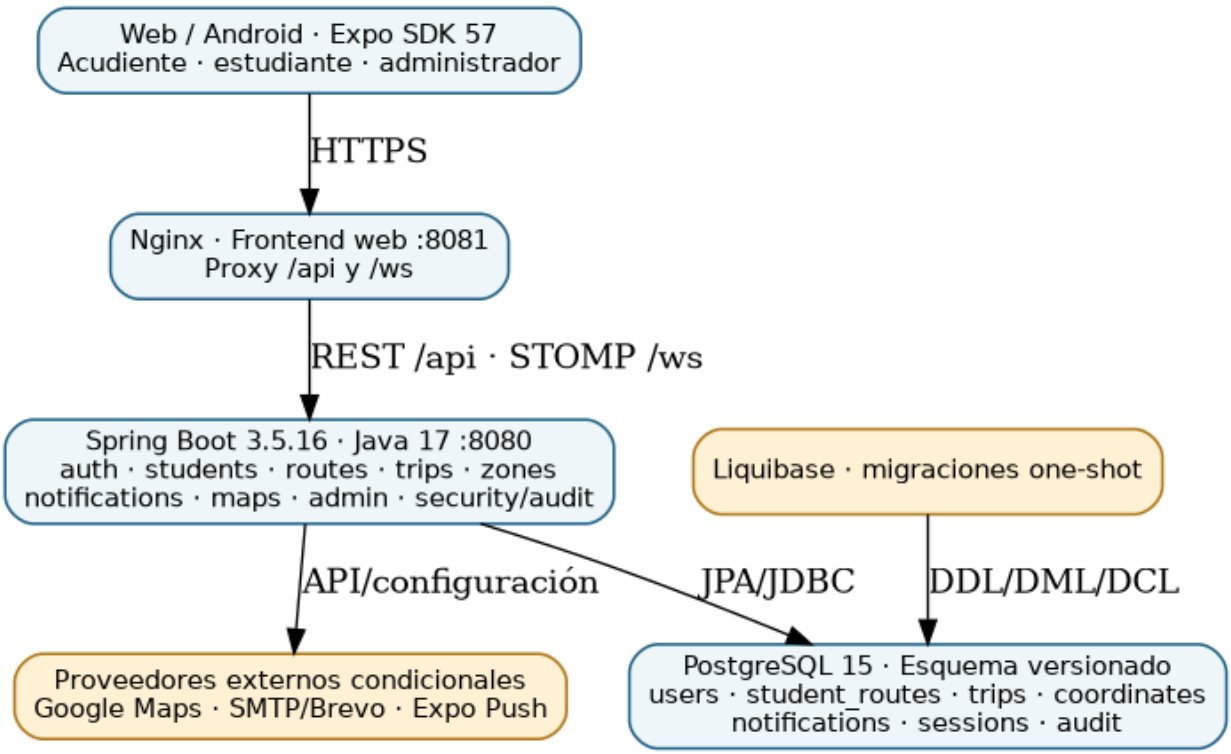


Figura 1. Componentes e intercambios del sistema

2. Escenario AV-01: registro, verificación y autenticación

Precondiciones: API accesible; correo habilitado si se exige entrega del mensaje; credenciales y política de contraseñas vigentes.

1. El cliente abre app/(auth)/register.js, valida el formulario y solicita POST /api/auth/register.
2. AuthController delega en servicios de autenticación, que validan datos y registran el usuario. La verificación utiliza un enlace/token de duración limitada.
3. El usuario confirma por GET /api/auth/verify-email cuando corresponde. La aplicación debe representar explícitamente el caso «correo aún no verificado».

4. POST /api/auth/login verifica credenciales, control de intentos y condiciones de dispositivo. Puede requerir confirmación de nuevo dispositivo y/o segundo factor; consultar confirm-login y rutas /2fa/*.

1. Con sesión válida, el cliente obtiene su perfil mediante GET /api/auth/me y adjunta Authorization: Bearer <jwt> a solicitudes protegidas.

2. En cierre de sesión el cliente invoca POST /api/auth/logout y elimina la credencial local según su plataforma.

Alternativas: credenciales erróneas, sesión revocada, demasiados intentos, email no entregado, token vencido y dispositivos no confirmados. **Evidencia de cierre:** request/response anonimizado, log de servicio, recepción del correo si se habilitó proveedor y pantalla de resultado. Ninguno de estos resultados se certifica aquí.

3. Escenario AV-02: estudiante, acudiente y vinculación QR

1. Un usuario con permiso STUDENT_CREATE o rol ADMIN registra al estudiante por POST /api/students.

2. Si hay autorización, se configuran acudientes POST /api/students/{studentId}/guardians y contactos de emergencia; al revocar acceso se usa DELETE /api/students/{studentId}/guardians/{guardianUserId}.

3. Un gestor autorizado solicita POST /api/students/{studentId}/link-code y comparte el QR/código únicamente con el dispositivo correspondiente.

4. La app móvil abre scan-qr.js o la pantalla de vínculo, presenta la información para revisión y solicita POST /api/students/link-device.

5. El backend valida el código, la identidad del dispositivo y la autorización correspondiente antes de vincular. La consulta GET /api/students/child-profile exige autoridad CHILD_DEVICE.

6. El acudiente comprueba la vinculación desde la ficha; para desvincular solicita DELETE /api/students/{studentId}/device.

Fallos relevantes: código caducado, reutilizado o incorrecto, cámara sin permiso, vínculo de otro estudiante, red desconectada, error de validación. El código no debe compartirse por canales públicos. **Control:** no confundir la confirmación visual del cliente con la autorización final del servidor.

4. Escenario AV-03: asignar ruta y establecer ruta preferida

Una ruta y sus paradas se gestionan en /api/routes; el permiso operativo se aplica por controlador. Para asociarla a un estudiante se usa POST /api/students/{studentId}/routes/{routeId} y para designar la preferida PUT /api/students/{studentId}/preferred-route/{routeId}. La migración 014 agrega student_routes.is_preferred e índice de unicidad parcial para asignaciones activas. Si el cliente crea un trayecto sin routeId, TripService.create busca primero una ruta preferida activa; en ausencia de preferida admite la única ruta activa disponible. Si hay varias rutas activas sin selección inequívoca, el backend rechaza la operación y exige elección.

Resultado esperado observable: 201 Created para un trayecto correctamente creado; para una ruta no asignada o ambigüa, mensaje de validación y ausencia de nuevo trayecto. La validez de la migración se verifica sobre la base realmente desplegada, no solo por la presencia del archivo SQL.

5. Escenario AV-04: trayecto GPS y comunicación en tiempo real

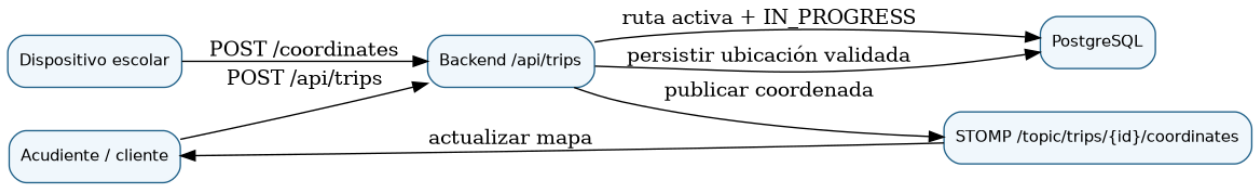


Figura 2. Flujo principal de trayecto y coordenadas

1. Un principal autorizado invoca POST /api/trips. El servicio impide simultaneidad de otro IN_PROGRESS para el mismo estudiante; el nuevo viaje inicia en ese estado.
2. El dispositivo infantil solicita permiso de ubicación en la plataforma; el cliente envía POST /api/trips/{tripId}/coordinates cuando el viaje está activo.
3. TripService verifica acceso a estudiante, estado del trayecto y saltos imposibles; registra la coordenada admisible y pasa el dato a TripAlertService.
4. El servidor difunde a /topic/trips/{tripId}/coordinates; una sesión STOMP suscrita actualiza la interfaz de seguimiento.
5. Los listados GET /api/trips y GET /api/trips/{tripId}/coordinates exponen paginación mediante page y size (valores predeterminados del controlador: página 0, tamaño 20).
6. Para dar de baja el trayecto, el gestor autorizado utiliza DELETE /api/trips/{tripId}; se marca deleted_at. Si estaba en curso, el servicio cambia el estado a CANCELED y registra la hora de cierre.

Límite confirmado: el controlador TripController de main no publica operación PATCH de transición de estados. La enumeración de dominio incluye PENDING, IN_PROGRESS, COMPLETED, CANCELED, pero ello no demuestra una interfaz de finalización de COMPLETED expuesta al usuario. No documentar un botón funcional sin prueba y endpoint concordantes.

6. Escenario AV-05: ubicación obsoleta y notificación

LocationAvailabilityService.checkActiveTrips se programa con guardian.gps.location-check-ms (valor por defecto **30000 ms**) y compara la ubicación más reciente con guardian.gps.location-stale-seconds (**120 s** por defecto). Para cada trayecto IN_PROGRESS cuya última coordenada sea antigua, identifica el usuario propietario y guardianes activos y solicita una notificación LOCATION_UNAVAILABLE evitando duplicados respecto del instante conocido. **No equivale a confirmación de entrega del mensaje push o correo**; eso depende de configuración, dispositivos registrados y servicios externos.

Pruebas imprescindibles: tiempo simulado 119/120/121 s, disparo programado, desduplicación, múltiples guardianes, GPS detenido, retorno de señal y revisión del canal de notificación en dispositivo real.

7. Escenarios AV-06 a AV-08: zonas seguras, mapas y notificaciones

Zonas seguras. El módulo /api/safe-zones gestiona geocercas por estudiante. La persistencia de un radio positivo y las alertas deben cotejarse en pruebas con coordenadas conocidas; visualizar el círculo en el mapa no demuestra que la alerta se envió.

Mapas. La UI consulta /api/maps/**, donde el servidor opera como proxy hacia Google Maps. Se aplican controles de cuota, caché y resiliencia configurados; una credencial ausente, cuota agotada o respuesta externa fallida requiere manejo explícito del error y no puede exponerse la clave privada.

Notificaciones. Se registran datos de notificaciones y recibos, se consultan desde /api/notifications y se cambia el estado de lectura vía /api/notifications/{receiptId}/read. Los canales externos (correo, Expo Push y SMS si estuviera configurado) no se certifican por la existencia de código.

8. Matriz de condiciones excepcionales

Código	Situación	Comportamiento esperado	Prueba requerida
EX-01	JWT ausente/invalidado	401 y sin datos privados	API sin token
EX-02	Usuario sin permiso	403 y sin mutación	Principal sin autorización
EX-03	Vinculación a estudiante ajeno	Denegación	Dispositivo de otro perfil
EX-04	Ruta no asignada	Rechazo del viaje	Crear con routeId inválido
EX-05	GPS obsoleto	Aviso persistido según umbral	Reloj y coordenadas controladas
EX-06	Salto GPS imposible	No persistir posición inválida	Coordenadas sintéticas
EX-07	Desconexión STOMP	Mostrar pérdida/reconexión	Cortar conectividad
EX-08	Falla proveedor externo	Error controlado y registro	Credencial de prueba / simulación
EX-09	Baja de viaje activo	CANCELED + ended_at + deleted_at	Lectura antes/después

9. Diagrama de estados y pruebas de consistencia

El modelo contiene estados PENDING, IN_PROGRESS, COMPLETED y CANCELED. **Transiciones verificadas en el servicio inspeccionado:** creación → IN_PROGRESS; eliminación autorizada de un viaje activo → CANCELED más baja lógica. Cualquier otra transición deberá respaldarse con otro controlador/servicio, commit y prueba antes de incorporarla como función disponible.

Se recomienda revisar los PDF/Draw.io históricos del ZIP recibido: los modelos de secuencia y estados 1.0/2.0 son antecedentes, no evidencia suficiente de que cada mensaje y acción exista en el código del release. La presente edición reemplaza los textos obsoletos sin sustituir todos los diagramas editables históricos.

10. Criterios de aceptación de la vista dinámica

La documentación puede considerarse alineada con el código cuando cada mensaje se ubica en un controlador real, los permisos se comparan con @PreAuthorize, las tablas se vinculan a migraciones, el caso positivo y negativo tienen escenario y el repositorio identifica un SHA reproducible. La validación de funcionamiento exige adjuntar pruebas y evidencia de ambiente. Estos dos criterios deben registrarse por separado.

Referencias y trazabilidad (APA 7.ª edición)

- GPS Guardian Escolar. (2026a). *Backend-Guardian-Escolar* [Código fuente, rama main, commit 1022058d]. GitHub. https://github.com/GPS-Guardian-Escolar/Backend-Guardian-Escolar/tree/1022058d545a9e7936edbff75950034ec4f98e88
- GPS Guardian Escolar. (2026b). *Fronted-GuardianEscolar-* [Código fuente, rama main, commit 3d9f59fa]. GitHub. https://github.com/GPS-Guardian-Escolar/Fronted-GuardianEscolar-/tree/3d9f59fa3e91b5ecc5b8ae86591187546b966668
- GPS Guardian Escolar. (2026c). *DB-Structure-Guardian-Escolar* [Migraciones SQL, rama main, commit 9278b1dc]. GitHub. https://github.com/GPS-Guardian-Escolar/DB-Structure-Guardian-Escolar/tree/9278b1dcb224301536d6158357f74daee74b20ba
- GPS Guardian Escolar. (2026d). *Docker-Infra-Guardian-Escolar* [Configuración de despliegue, rama main, commit 131e9895]. GitHub. https://github.com/GPS-Guardian-Escolar/Docker-Infra-Guardian-Escolar/tree/131e98954430e95e9108174eded9896b3f0d5272
- GPS Guardian Escolar. (2026e). *Docs-Guardian-Escolar: entregables 2.5.5 y suplementos 2.5.6* [Documentación del proyecto]. GitHub. https://github.com/GPS-Guardian-Escolar/Docs-Guardian-Escolar

Fuentes primarias relevantes: TripController.java, TripService.java, StudentController.java, LocationAvailabilityService.java, WebSocketConfig.java, application.yml, package.json, docker-compose.yaml y 01_ddl/04_alter/014_preferred_student_route.sql, según el documento. Las referencias a rutas son comprobables en los repositorios indicados.